

Package ‘GlycoDiveR’

June 18, 2026

Title Visualization of glycoproteomic data

Version 1.3.0

Description GlycoDiveR generates graphics from quantitative glycoproteomics datasets directly from search engine output files. It currently supports FragPipe, Byonic, pGlyco, Perseus, and MSstats output files. The package converts the data into a generic GlycoDiveR format that incorporates key experimental metadata along with UniProt-derived metadata. The GlycoDiveR formatted input is directly compatible with all GlycoDiveR graphics functions.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Imports circlize,
ComplexHeatmap,
plyr,
dplyr,
ggplot2,
ggrepel,
magrittr,
patchwork,
seqinr,
tidyr,
utils,
igraph,
data.table,
stringr,
UpSetR,
rlang,
purrr,
tidyselect,
readxl,
httr,
scales,
ggnewscale,
impute

Depends R (>= 4.1.0)

Suggests knitr,

rmarkdown,
testthat ($\geq 3.0.0$),
devtools

Config/testthat/edition 3

LazyData true

VignetteBuilder knitr

Contents

ComputeMissingness	3
ComputeNumberOfGlycoforms	4
exampleData	5
FilterComparison	6
GetAnnotationTemplate	7
GetComparison	7
ImportByonic	8
ImportGlycanFinder	9
ImportMSFragger	11
ImportMSstatsAbundances	13
ImportMSstatsComparison	13
ImportPerseusAbundances	14
ImportPerseusComparison	15
ImportpGlyco	15
ManageColorScheme	17
ManageGlycanCategories	17
ManageGlycanColors	18
ManageGlycanDatabase	19
ManageModificationDatabase	20
PlotComparisonVolcano	21
PlotCompletenessMatrix	22
PlotCV	23
PlotErrorVersusRt	24
PlotGlycanCompositionBar	26
PlotGlycanCompositionPie	27
PlotGlycansPerSite	28
PlotGlycanSubcellularLocation	29
PlotGlycopeptideCount	30
PlotGlycoproteinCount	31
PlotGlycoProteinGlycanNetwork	32
PlotGlycoproteinRank	34
PlotGlycoPSMCount	35
PlotGlycositeIdsPerRun	36
PlotGlycositesVsGlycans	37
PlotPCA	38
PlotPeptideLength	40
PlotPSMCount	41
PlotPSMsVsTime	42
PlotPTMQuantification	43
PlotQuantificationQC	45
PlotSiteGlycanComposition	46
PlotSiteGlycanCompositionBar	47

PlotSiteQuantification	48
PlotUpSet	49
PlotVennDiagram	51

Index	53
--------------	-----------

ComputeMissingness	<i>ComputeMissingness</i>
--------------------	---------------------------

Description

This computes the percentage of missing values in your GlycoDiveR uploaded data. The computation is done on a (glyco)peptide level. Peptides with a quantitative value of 0 are considered missing.

Usage

```
ComputeMissingness(
  input,
  type = "glyco",
  removeAllNA = TRUE,
  whichPeptide = NULL,
  whichAlias = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

input	Your formatted GlycoDiveR data.
type	Choose between "glyco" or "all"
removeAllNA	Remove peptides with only NA values. This can, for example happen if you upload abundance values using Perseus or MSstats.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichAlias	provide a vector of Aliases to only select these aliases for plotting
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
silent	silence printed information (default = FALSE)

Value

The percentage of missing data.

Examples

```
## Not run: ComputeMissingness(mydata, type = "glyco")
```

```
ComputeNumberOfGlycoforms
```

```
ComputeNumberOfGlycoforms
```

Description

ComputeNumberOfGlycoforms

Usage

```
ComputeNumberOfGlycoforms(
  input,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  whichAlias = NULL,
  whichPeptide = NULL,
  silent = FALSE
)
```

Arguments

input	Formatted data
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
silent	silence printed information (default = FALSE)

Value

The number of potential glycoforms.

Examples

```
## Not run:  
ComputeNumberOfGlycoforms(mydata, whichProtein = "Q9NZQ7")  
## End(Not run)
```

exampleData

Example dataset for GlycoDiver

Description

A small example dataset used in package examples and testing.

Usage

exampleData

Format

A GlycoDiver dataset (list) with 10 items:

PSMTable The PSM table (data frame)

rawPSMTable The raw PSM table (data frame)

PTMTable The PTM table (data frame)

annotation The annotation file (data frame)

searchEngine The search engine (character)

peptideScoreCutoff Score cutoff (numeric)

glycanScoreCutoff Score cutoff (numeric)

filterForNoNSequon Score cutoff (numeric)

confidenceLevels Score cutoff (numeric)

deltaModCutoff Score cutoff (numeric)

Source

Generated for package examples

FilterComparison

*FilterComparison***Description**

This functions helps filter your comparison dataframe, which can subsequently be used for other plots and functions. The dataframe needs to have a column supplied by the `whichStatistic` argument (typically, "pvalue", or "adjpvalue"). Alternatively, you can filter the comparison dataframe yourself, for example, using `subset` or `dplyr::filter`.

Usage

```
FilterComparison(
  comparison,
  statisticalCutoff = 0.05,
  whichStatistic = "pvalue",
  whichLabel = NA,
  minf = FALSE,
  inf = FALSE
)
```

Arguments

<code>comparison</code>	A comparison dataframe.
<code>statisticalCutoff</code>	What statistical cutoff to use (default = 0.05)
<code>whichStatistic</code>	Typically, "pvalue", or "adjpvalue". Must correspond to an identically named column in your comparison dataframe.
<code>whichLabel</code>	What label(s) to use. Corresponds to the Label column of the input comparison dataframe.
<code>minf</code>	TRUE/FALSE if -inf should be retained (default = FALSE).
<code>inf</code>	TRUE/FALSE if inf should be retained (default = FALSE)

Value

A subset of your comparison dataframe

Examples

```
## Not run: FilterComparison(comparison, whichLabel =
c("SampleA-SampleB", "SampleB-SampleC"))
## End(Not run)
```

GetAnnotationTemplate *GetAnnotationTemplate*

Description

This created an annotation.csv file in the path you specify.

Usage

```
GetAnnotationTemplate(path, tool)
```

Arguments

path	The path the the folder that holds the subfolder with the PSM files
tool	The search engine used. Options: MSFragger, Byonic, pGlyco

Value

Exported annotation file

Examples

```
## Not run: GetAnnotationTemplate("Z:/Subfolder", tool = "MSFragger")
```

GetComparison *GetComparison*

Description

Computes pvalues, BH-corrected pvalues, and log2 fold changes for the specified conditions. Comparisons are done on the glycopeptide level using the max intensity value for duplicate PSMs.

Usage

```
GetComparison(input, comparisons, type = "glyco", whichAlias = NULL)
```

Arguments

input	Your already imported GlycoDiveR data.
comparisons	the comparisons you would like to include
type	Choose "glyco" to use only glycopeptides. Otherwise choose "all".
whichAlias	provide a vector of Aliases to only select these aliases for plotting

Value

a comparison dataframe

Examples

```
## Not run: GetComparison(mydata, list(c("PBS", "Lis")))
```

ImportByonic

ImportByonic

Description

This is the importer for Byonic data. It needs to untouched Byonic output, which is a Excel file with at least a Summary and a Spectra tab. This script will look for all files in the folder and all subfolders using the name tag "Byonic.xlsx".

Usage

```
ImportByonic(
  path,
  annotation,
  fastaPath,
  peptideScoreCutoff = 0,
  glycanScoreCutoff = 0.01,
  deltaModCutoff = 1,
  scrape = TRUE,
  removeReverse = TRUE,
  cutoffFilter = TRUE,
  dropNoQuant = FALSE,
  minPeptideCoverage = FALSE,
  thresholdMode = c("group", "total"),
  GlyToucan = TRUE
)
```

Arguments

path	The folder path, which will be used to find all "Byonic.xls" files in this folder and all subfolders.
annotation	The annotation dataframe. Please generate a template using GetAnnotationTemplate("path", tool = "Byonic")
fastaPath	The path to the fasta file that was used.
peptideScoreCutoff	The score cutoff from the "Score" column. The cutoff is the lower limit.
glycanScoreCutoff	The score cutoff for the Log Prob column. The Log Prob column is converted using $10^{(-\text{Log Prob})}$. The cutoff is the upper limit.
deltaModCutoff	The score cutoff for the Delta Mod score. The score is the lower limit.
scrape	set TRUE/FALSE to use scraping of Uniprot data.
removeReverse	Remove reverse hits as noted with the ">Reverse " tag.
cutoffFilter	Filter for the glycan score, peptide score, N Sequon, and confidence levels.
dropNoQuant	Remove or keep PSMs without a quantitative value or have a quant of 0.
minPeptideCoverage	Works together with the thresholdMode argument. If thresholdMode = "total", provide a single numeric value. Values between 0 and 1 are interpreted as proportions (e.g., 0.1 = 10%), while values ≥ 1 are interpreted as absolute counts

	(e.g., 2 = at least two peptides identified). If thresholdMode = "group", provide a numeric vector of length two (e.g., c(0.1, 3)). The first value is interpreted in the same way as for "total" (proportion or absolute count). The second value specifies the minimum number of groups in which this threshold must be met.
thresholdMode	Character string specifying the filtering mode: "total" applies a global threshold across all data, while "group" identifies the threshold within groups.
GlyToucan	set to TRUE will connect to the GlyCosmos API to retrieve GlyToucan identifiers.

Value

Formatted GlycoDiveR data file.

Examples

```
## Not run: ImportByonic(path = "C:/Byonic",
  annotation = "C:/annotation.csv",
  fastaPath = "C:/fastafile.fasta",
  peptideScoreCutoff = 0,
  glycanScoreCutoff = 1)
## End(Not run)
```

ImportGlycanFinder	<i>ImportGlycanFinder</i>
--------------------	---------------------------

Description

This imports search results output by GlycanFinder. It will work for N-glycopeptide searches and O-glycopeptides searches. During import, this function will look for all the glycan.glycopsm.csv files in the specified folder and all the subfolders.

Usage

```
ImportGlycanFinder(
  path,
  annotation,
  fastaPath,
  peptideScoreCutoff = 0,
  glycanScoreCutoff = 0,
  scrape = TRUE,
  normalization = "median",
  cutoffFilter = TRUE,
  dropNoQuant = FALSE,
  minPeptideCoverage = FALSE,
  thresholdMode = c("group", "total"),
  useExtendedOGlycanCategories = FALSE,
  GlyToucan = TRUE,
  SScoreCutoff = FALSE,
  structureConfidenceCutoff = FALSE,
  AScoreCutoff = FALSE
)
```

Arguments

path	Path to search engine output folder
annotation	Path to annotation file
fastaPath	Path to FASTA file
peptideScoreCutoff	Peptide score cutoff, which is the Glycopeptide Score column in the search engine output file.
glycanScoreCutoff	Glycan score cutoff, which is the Glycan Score column in the search engine output file.
scrape	set TRUE/FALSE to use scraping of Uniprot data.
normalization	The (glyco)peptide normalization used. Choose between "median" (default) or "none".
cutoffFilter	Filter for the glycan score, peptide score, and confidence levels during data import
dropNoQuant	Remove or keep PSMs without a quantitative value or have a quant of 0.
minPeptideCoverage	Works together with the thresholdMode argument. If thresholdMode = "total", provide a single numeric value. Values between 0 and 1 are interpreted as proportions (e.g., 0.1 = 10%), while values ≥ 1 are interpreted as absolute counts (e.g., 2 = at least two peptides identified). If thresholdMode = "group", provide a numeric vector of length two (e.g., c(0.1, 3)). The first value is interpreted in the same way as for "total" (proportion or absolute count). The second value specifies the minimum number of groups in which this threshold must be met.
thresholdMode	Character string specifying the filtering mode: "total" applies a global threshold across all data, while "group" identifies the threshold within groups.
useExtendedOGlycanCategories	set to FALSE will classify all O-glycans as OGlycan Set to TRUE will classify them in "Sialofucosylated", "Sialylated", "Fucosylated", and "OGlycan".
GlyToucan	set to TRUE will connect to the GlyCosmos API to retrieve GlyToucan identifiers.
SScoreCutoff	The lower limit of the S score used to filter the data, e.g. SScoreCutoff = 10. This will retain S Scores that are 10 or higher.
structureConfidenceCutoff	The Structure Cutoffs to be retained. For example, structureConfidenceCutoff = c("LOW", "HIGH")
AScoreCutoff	The lower limit of the A score used to filter the data, e.g. AScoreCutoff = 10. This will retain A Scores 10 or higher. The highest score is retained for multiply glycosylated peptides.

Value

A formatted GlycoDiveR file.

Examples

```
## Not run:
ImportGlycanFinder(
  path = "Z:/Analysis 2",
```

```

    annotation = "Z:/annotation.csv",
    fastaPath = "Z:/glycan.proteins.fasta",
    normalization = "none",
    GlyToucan = FALSE,
    structureConfidenceCutoff = "HIGH",
    AScoreCutoff = 10
)

## End(Not run)

```

ImportMSFragger

ImportMSFragger

Description

This imports search results output by MSFragger/Fragpipe. It will work for N-glycopeptide searches and O-glycopeptides searches with or without OPair. During import, this function will look for all the psm.tsv files in the specified folder and all the subfolders. It will also use the combined_modified_peptide.tsv if you choose to use FragPipe computed normalized intensities, or the abundance_multi-mass_MD.tsv files whenever FP_TMT normalization is selected, note: this only works for Glyco-TMT data.

Usage

```

ImportMSFragger(
  path,
  annotation,
  fastaPath,
  peptideScoreCutoff = 0,
  glycanScoreCutoff = 0.01,
  scrape = TRUE,
  normalization = "median",
  convertFPMoCodeToMass = TRUE,
  filterForNoNSequon = FALSE,
  confidenceLevel = FALSE,
  TMT = FALSE,
  cutoffFilter = TRUE,
  dropNoQuant = FALSE,
  minPeptideCoverage = FALSE,
  thresholdMode = c("group", "total"),
  useExtendedOGlycanCategories = FALSE,
  GlyToucan = TRUE
)

```

Arguments

path	Path to search engine output folder
annotation	Path to annotation file
fastaPath	Path to FASTA file
peptideScoreCutoff	Peptide score cutoff, which is the Hyperscore column in the search engine output file.

glycanScoreCutoff	Glycan score cutoff, which is the Glycan Q Value column in the search engine output file.
scrape	set TRUE/FALSE to use scraping of Uniprot data.
normalization	The (glyco)peptide normalization used. Choose between "median" (default), FP_Normalized, FP_MaxLFQ, FP_TMT, or none. median: performs median normalization. FP_Normalized: extracts the intensity values in the Intensity columns of the combined_modified_peptide.tsv files. FP_MaxLFQ: extracts the intensity values in the MaxLFQ.Intensity columns of the combined_modified_peptide.tsv files. FP_TMT: Uses the abundance_multi-mass_MD.tsv files to import the FragPipe calculated normalized intensities. none: uses the raw intensity values from the PSM.tsv files.
convertFPMoCodeToMass	MSFragger uses modification code in peptide modified sequences. This replaces the code with the mass of the modification. Keep this enabled when importing MSstats comparison results as MSstats uses modification masses instead of modification codes. Default = TRUE
filterForNoNSequon	Filter for peptides without an N-sequon. Only works for OPair data (default = FALSE).
confidenceLevel	What OPair confidence levels to accept, options are Level1, Level1b, Level2, Level3 (default = FALSE). Provide like this: confidenceLevel = c("Level1", "Level1b")
TMT	set TRUE or FALSE to select for a TMT experiment.
cutoffFilter	Filter for the glycan score, peptide score, N Sequon, and confidence levels.
dropNoQuant	Remove or keep PSMs without a quantitative value or have a quant of 0.
minPeptideCoverage	Works together with the thresholdMode argument. If thresholdMode = "total", provide a single numeric value. Values between 0 and 1 are interpreted as proportions (e.g., 0.1 = 10%), while values >= 1 are interpreted as absolute counts (e.g., 2 = at least two peptides identified). If thresholdMode = "group", provide a numeric vector of length two (e.g., c(0.1, 3)). The first value is interpreted in the same way as for "total" (proportion or absolute count). The second value specifies the minimum number of groups in which this threshold must be met.
thresholdMode	Character string specifying the filtering mode: "total" applies a global threshold across all data, while "group" identifies the threshold within groups.
useExtendedOGlycanCategories	set to FALSE will classify all O-glycans as OGlycan Set to TRUE will classify them in "Sialofucosylated", "Sialylated", "Fucosylated", and "OGlycan".
GlyToucan	set to TRUE will connect to the GlyCosmos API to retrieve GlyToucan identifiers.

Value

Formatted GlycoDiveR data file.

Examples

```
## Not run: ImportMSFragger(path = "Z:/Folder",
```

```

annotation = "Z:/Folder/annotation.csv",
fasta = "Z:/fasta.fasta",
peptideScoreCutoff = 0,
glycanScoreCutoff = 0.05,
scrape = FALSE)
## End(Not run)

```

ImportMSstatsAbundances

ImportMSstatsAbundances

Description

This script uses MSstats generated quantitative data and uses this data to fill in the existing quantitative values. It will only retain quantitative values present in the MSstats data. Requirements are 1) matching Alias column with the same values as in your GlycoDiveR data, 2) MSstats data has a LogIntensities column, 3) MSstats data has a ModifiedPeptide column with values corresponding to the ModifiedPeptide column in your data. This script is for MSstats data, but should work for other inputs too.

Usage

```
ImportMSstatsAbundances(input, MSstatsPath, cleanCCarbamidomethylation = TRUE)
```

Arguments

input	This is your formatted GlycoDiveR data
MSstatsPath	The filepath to the MSstats generated csv.
cleanCCarbamidomethylation	Removes \[57.0215\] from the MSstats ModifiedPeptide column (default = TRUE)

Value

your input data with updated quantitative values.

Examples

```
## Not run: ImportMSstatsComparison(mydata, "C:/User/Pathto csv.csv")
```

ImportMSstatsComparison

ImportMSstatsComparison

Description

This imports the MSstats comparison results. It needs the following columns to work fully: Protein, PeptideSequence, Label, log2FC, pvalue, adj.pvalue.

Usage

```
ImportMSstatsComparison(path, input = NULL, cleanCCarbamidomethylation = TRUE)
```

Arguments

path	The filepath to the MSstats output file.
input	Your already imported GlycoDiveR data.
cleanCCarbamidomethylation	Removes \[57.0215] from the MSstats ModifiedPeptide column (default = TRUE)

Value

A comparison table with the MSstats comparison results

Examples

```
## Not run: ImportMSstatsComparison("C:/MSstatsFile.csv")
```

ImportPerseusAbundances

ImportPerseusAbundances

Description

ImportPerseusAbundances

Usage

```
ImportPerseusAbundances(input, PerseusPath, cleanCCarbamidomethylation = TRUE)
```

Arguments

input	GD data
PerseusPath	path
cleanCCarbamidomethylation	yes

Value

Dataframe

Examples

```
## Not run:
ImportPerseusAbundances(mydata, PerseusPath = "C:/topath")

## End(Not run)
```

ImportPerseusComparison

ImportPerseusComparison

Description

This imports a Perseus comparison and returns it as a dataframe. The Conditions specified in Perseus must match the Condition values in the GlycoDiveR annotation file. Perseus formats column names as such: Condition1_Condition2. GlycoDiveR creates identical mapping.

Usage

```
ImportPerseusComparison(input, path, cleanCCarbamidomethylation = TRUE)
```

Arguments

input	Your already imported GlycoDiveR data.
path	The path to the Perseus .txt file.
cleanCCarbamidomethylation	Removes \[57.0215\] from the MSstats ModifiedPeptide column (default = TRUE)

Value

A comparison dataframe.

Examples

```
## Not run: ImportPerseusComparison(mydata, C:/PerseusOutput.txt)
```

ImportpGlyco

ImportpGlyco

Description

This is the importer for pGlyco data. It needs to untouched pGlyco output, which is a .txt file named 'pGlycoDB-GP-FDR-Pro-Quant-Site.txt'. This script will look for all files in the folder and all subfolders.

Usage

```
ImportpGlyco(
  path,
  annotation,
  fastaPath,
  peptideScoreCutoff = 0.01,
  glycanScoreCutoff = 0.01,
  normalization = "median",
  scrape = TRUE,
  cutoffFilter = TRUE,
```

```

dropNoQuant = FALSE,
minPeptideCoverage = FALSE,
thresholdMode = c("group", "total"),
GlyToucan = TRUE
)

```

Arguments

path	The folder path, which will be used to find all 'pGlycoDB-GP-FDR-Pro-Quant-Site.txt' files in this folder and all subfolders.
annotation	The annotation dataframe. Please generate a template using GetAnnotationTemplate("path", tool = "pGlyco")
fastaPath	The path to the fasta file that was used.
peptideScoreCutoff	The score cutoff for the "PeptideFDR" column. The cutoff is the upper limit.
glycanScoreCutoff	The score cutoff for the "GlycanFDR" column. The score is the upper limit.
normalization	The (glyco)peptide normalization used. Choose between "none" or "median" (default = "median").
scrape	set TRUE/FALSE to use scraping of Uniprot data.
cutoffFilter	Filter for the glycan score, peptide score, N Sequon, and confidence levels.
dropNoQuant	Remove or keep PSMs without a quantitative value or have a quant of 0.
minPeptideCoverage	Works together with the thresholdMode argument. If thresholdMode = "total", provide a single numeric value. Values between 0 and 1 are interpreted as proportions (e.g., 0.1 = 10%), while values >= 1 are interpreted as absolute counts (e.g., 2 = at least two peptides identified). If thresholdMode = "group", provide a numeric vector of length two (e.g., c(0.1, 3)). The first value is interpreted in the same way as for "total" (proportion or absolute count). The second value specifies the minimum number of groups in which this threshold must be met.
thresholdMode	Character string specifying the filtering mode: "total" applies a global threshold across all data, while "group" identifies the threshold within groups.
GlyToucan	set to TRUE will connect to the GlyCosmos API to retrieve GlyToucan identifiers.

Value

Formatted GlycoDiveR data file.

Examples

```

## Not run: ImportpGlyco(path = "Z:/Folder",
annotation = "Z:/Folder/annotation.csv",
fasta = "Z:/fasta.fasta",
peptideScoreCutoff = 0.01,
glycanScoreCutoff = 0.01,
scrape = FALSE)
## End(Not run)

```

ManageColorScheme	<i>ManageColorScheme</i>
-------------------	--------------------------

Description

Manage the color palette used for plotting

Usage

```
ManageColorScheme(  
  edit = NULL,  
  action = c("View", "Append", "Replace", "Reset")  
)
```

Arguments

edit	Supply a vector containing R supported colors. These can for example be "white" or "#E1D89F".
action	Choose between the following: View: to view the current modification database. Append: append to the current modification database with the supplied edit. Replace: replace the modification database with the supplied edit. Reset: reset to the default values.

Value

The updated database.

Examples

```
## Not run:  
ManageColorScheme(edit = c("white", "black", "#E1D89F"), action="Append")  
  
## End(Not run)
```

ManageGlycanCategories	<i>Manage glycan categories</i>
------------------------	---------------------------------

Description

Manage the glycan categories used by GlycoDiveR. Run ManageGlycanCategories(action = "View") to see what this database looks like. To use custom glycan categories, please run this function before you import your data. The colors for new glycan categories should be registered using ManageGlycanColors().

Usage

```
ManageGlycanCategories(  
  edit = NULL,  
  action = c("View", "Replace", "Reset", "showCode")  
)
```

Arguments

<code>edit</code>	A data frame used to replace the existing glycan categories. You can generate a template for this data frame using <code>action = "showCode"</code>. The columns for the dataframe are: Priority Numeric priority used to resolve matches. The category with the highest priority is assigned. Residue Amino acid at the glycosylation site. Pattern Pattern matched against the <code>TotalGlycanComposition</code> column (see <code>mydata\$PSMTable\$TotalGlycanComposition</code>). The pattern is a Regular Expressions (Regex). AI is super helpful getting the Regex that you need. Min_H / Max_H Minimum and maximum number of hexoses. Min_N / Max_N Minimum and maximum number of HexNAc residues. Category Assigned glycan category for the highest-priority match. If you introduce new glycan categories, use <code>ManageGlycanColors()</code> to assign colors to them.
<code>action</code>	Character string specifying the action to perform. One of: "View" Return the current glycan categories. "Replace" Replace the current glycan categories with <code>edit</code>. "Reset" Reset glycan categories to the default values. "showCode" Return example code for creating a new <code>edit</code> data frame.

Value

A data frame containing the (updated) glycan categories.

Examples

```
## Not run:
# View current categories
ManageGlycanCategories(action = "View")

# Replace categories using a template
ManageGlycanCategories(
  edit = df_from_action_equals_showCode,
  action = "Replace"
)

## End(Not run)
```

ManageGlycanColors *ManageGlycanColors*

Description

Manage the color palette used for glycans.

Usage

```
ManageGlycanColors(  
  edit = NULL,  
  action = c("View", "Append", "Replace", "Reset")  
)
```

Arguments

edit	Supply a path to a csv file containing the headers "GlycanType", and "color", or supply a data frame with those headers.
action	Choose between the following: View: to view the current modification database. Append: append to the current modification database with the supplied edit. If duplicate GlycanType values are found, it will overwrite the existing values. Replace: replace the modification database with the supplied edit. Reset: reset to the default values.

Value

The updated database.

Examples

```
## Not run:  
ManageGlycanColors(edit = mydataframe, action="Append")  
  
## End(Not run)
```

ManageGlycanDatabase *ManageGlycanDatabase*

Description

Manage the glycan database.

Usage

```
ManageGlycanDatabase(  
  edit = NULL,  
  action = c("View", "Append", "Replace", "Reset")  
)
```

Arguments

edit	Supply a path to a csv file containing the headers "FullName", "ShortName" and "GMass", or supply a data frame with those headers.
action	Choose between the following: View: to view the current modification database. Append: append to the current modification database with the supplied edit. Replace: replace the modification database with the supplied edit. Reset: reset to the default values.

Value

The updated database.

Examples

```
## Not run:
ManageGlycanDatabase(edit = "C:/locationOfDataframe.csv", action="Append")

## End(Not run)
```

ManageModificationDatabase

ManageModificationDatabase

Description

Manage the modification database.

Usage

```
ManageModificationDatabase(
  edit = NULL,
  action = c("View", "Append", "Replace", "Reset")
)
```

Arguments

edit	Supply a path to a csv file containing the headers "FullName" and "ModificationMass", or supply a data frame with the headers "FullName" and "ModificationMass".
action	Choose between the following: View: to view the current modification database. Append: append to the current modification database with the supplied edit. Replace: replace the modification database with the supplied edit. Reset: reset to the default values.

Value

The updated database.

Examples

```
## Not run:
ManageModificationDatabase(edit = mydataframe, action="Append")

## End(Not run)
```

PlotComparisonVolcano *PlotComparisonVolcano*

Description

A volcano plot showing Log2 transformed intensity values and either p values or FDR values.

Usage

```
PlotComparisonVolcano(
  input,
  whichComparison,
  labelFormat = c("protein-site", "protein-site-composition"),
  statisticalCutoff = 0.05,
  log2FCCutoff = 1,
  statistic = "adjpvalue",
  whichLabel = "significant",
  plotColors = c("#88CCEE", "#BBBBBB", "#882255"),
  maxOverlaps = 10
)
```

Arguments

input	Formatted comparison dataframe (e.g. using GlycoDiveR's ImportMSstatsComparison function)
whichComparison	Which comparison as in the Label column.
labelFormat	Determine how the labels should be displayed
statisticalCutoff	What statistical cutoff.
log2FCCutoff	What log2 fold-change cutoff.
statistic	Choose "adjpvalue" or "pvalue".
whichLabel	What to label in the plot. choose "significant", "none", or supply a vector with the points to label. For example c("HPT-149", "SPA3K-N271"). These are internally generated using the comparison dataframe: paste(Proteins, ModificationID, sep = "-")
plotColors	The colors used in the plot.
maxOverlaps	The maximum number of overlapping labels in the volcano plot

Value

A volcano plot

Examples

```
## Not run:
PlotComparisonVolcano(comparison, whichComparison = "Sal-PBS",
  statistic = "pvalue", statisticalCutoff = 0.01)
## End(Not run)
```

PlotCompletenessMatrix

Plot Completeness Matrix

Description

Plot a matrix showing the identified and missed (glyco)peptides. Each row represents one unique glycopeptide.

Usage

```
PlotCompletenessMatrix(
  input,
  peptideType = "glyco",
  useMultiCategory = TRUE,
  plotColors = c("grey90", "#44AA99"),
  collapseTechReps = FALSE,
  showGlycanCounts = TRUE,
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  exportDataTo = FALSE,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
peptideType	Choose "glyco" to only view glycopeptide and "other" to also include non-modified peptides.
useMultiCategory	Set to TRUE shows multiple glycosylated peptides using the "Multi" glycan category. Set to FALSE separates multiple glycosylated peptides into each of the glycan categories.
plotColors	The colors used for the matrix The default is c("lightgrey", "darkgreen").
collapseTechReps	Do you want to collapse the technical replicates.
showGlycanCounts	TRUE to show the counts on the y-axis, or FALSE to not show the counts.
whichAlias	Provide a vector of Aliases to only select these aliases for plotting, e.g. whichAlias = c("Alias1", "Alias2").
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.

exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
exportDataTo	If a system folder is provided, a CSV file will be exported to that folder containing the data presented in the matrix
silent	silence printed information (default = FALSE)

Value

A matrix.

Examples

```
## Not run:
PlotCompletenessMatrix(mydata)
PlotCompletenessMatrix(mydata, peptideType = "other", silent = FALSE)

## End(Not run)
```

PlotCV	<i>PlotCV</i>
--------	---------------

Description

Plot the protein CVs using violing plots. The number of proteins in each group are labeled. Choose between glyco only, or all, and decide if you want to return a plot or the underlying data. The returned data can directly be used to filter other plots using the whichPeptide argument.

Usage

```
PlotCV(
  input,
  CV = c("base", "geometric"),
  horizontalLines = c(0, 20, 50, 100),
  whichProtein = NULL,
  type = c("all", "glyco"),
  returnType = c("plot", "data", "GDdata"),
  returnThreshold = NULL,
  whichPeptide = NULL,
  exactProteinMatch = TRUE,
  whichAlias = NULL,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
CV	Choose "base" to calculate CV using the non-normalized values, and "geometric" to calculate the geometric CV using log transformed values.
horizontalLines	What horizontal lines to show in the plot.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
type	Choose "glyco" or "all"
returnType	Choose "plot" to return the plot, "data" to return the CV data as a dataframe (this dataframe can be used to filter any visualization using the whichPeptide argument), or "GDdata" to return filtered GlycoDiveR data.
returnThreshold	Works in concert with returnType = "data". Defines the the upper CV threshold.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
silent	silence printed information

Value

A ggplot, dataframe, or GlycoDiveR data

Examples

```
## Not run:
PlotCV(mydata, type = "glyco", returnType = "plot")

PlotCV(mydata, type = "all", returnType = "GDdata", returnThreshold = 20)

## End(Not run)
```

PlotErrorVersusRt

PlotErrorVersusRt

Description

A scatter plot showing the retention time versus the m/z error. The three dotted lines represent the median, and the median plus or minus one standard deviation. The median and standard deviation calculations will use only points within the specified upper and lower limits.

Usage

```
PlotErrorVersusRt(
  input,
  type = c("both", "glyco", "nonGlyco"),
  upperLimit = NA,
  lowerLimit = NA,
  alpha = 0.5,
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

<code>input</code>	Formatted data imported through a GlycoDiveR importer.
<code>type</code>	Choose "both", "glyco", or "nonGlyco".
<code>upperLimit</code>	The upper m/z error limit.
<code>lowerLimit</code>	The lower m/z error limit.
<code>alpha</code>	The transparency of the dots. Set between 0 and 1.
<code>whichAlias</code>	Provide a vector of Aliases to only select these aliases for plotting.
<code>whichPeptide</code>	Filter what peptides to plot. This can either be a dataframe with a <code>ModifiedPeptide</code> peptide column, or a vector with the <code>ModifiedPeptide</code> sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the <code>FilterComparison</code> function.
<code>whichProtein</code>	Filter what proteins to plot. These are the IDs as presented in the <code>UniprotIDs</code> column in your GlycoDiveR data. This can either be a dataframe with a <code>UniprotIDs</code> column, or a vector with the <code>UniprotIDs</code> you want to keep.
<code>exactProteinMatch</code>	This is only relevant if you select for proteins using the <code>whichProtein</code> argument. When set to <code>TRUE</code> (default), your supplied <code>UniprotIDs</code> must be an exact match to the <code>UniprotIDs</code> in the dataframe. When set to <code>FALSE</code> , it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to <code>FALSE</code> .
<code>silent</code>	silence printed information.

Value

A ggplot graph.

Examples

```
## Not run:
PlotErrorVersusRt(mydata, type = "both")

PlotErrorVersusRt(mydata, type = "glyco", lowerLimit = -20, upperLimit = 20)

## End(Not run)
```

PlotGlycanCompositionBar

PlotGlycanCompositionBar

Description

Visualize the glycan composition per protein, multiple proteins, or all the proteins.

Usage

```
PlotGlycanCompositionBar(
  input,
  summaryFunction = "count",
  grouping = "condition",
  scales = "fill",
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
summaryFunction	Use "count" to use glycopeptide count, "intensity" to summarize by glycopeptide intensity.
grouping	Grouping is "all", "technicalReps", "biologicalReps", or "condition".
scales	Controls plot normalization, choose "fill" or "stack".
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
silent	silence printed information

Value

A barplot.

Examples

```
## Not run:
PlotGlycanCompositionBar(mydata, scales = "stack")

## End(Not run)
```

```
PlotGlycanCompositionPie
      PlotGlycanCompositionPie
```

Description

Visualize glycan compositions as a pie chart.

Usage

```
PlotGlycanCompositionPie(
  input,
  grouping = "condition",
  summaryFunction = "count",
  scales = "free",
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
grouping	Grouping is "technicalReps", "biologicalReps", or "condition".
summaryFunction	Use "count" to use glycopeptide count, "intensity" to summarize by glycopeptide intensity.
scales	Controls plot normalization, choose "fixed" or "free".
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
silent	silence printed information (default = FALSE)

Value

The glycan composition

Examples

```
## Not run:
PlotGlycanCompositionPie(mydata)

PlotGlycanCompositionPie(mydata, grouping = "technicalReps",
                        whichProtein = c("P10204", "Q92930"))

## End(Not run)
```

PlotGlycansPerSite	<i>PlotGlycanPerSite</i>
--------------------	--------------------------

Description

This plot shows how many glycans were detected at each site— that is, how many sites have 1 glycan, 2 glycans, and so on.

Usage

```
PlotGlycansPerSite(
  input,
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

<code>input</code>	Formatted data imported through a GlycoDiveR importer.
<code>whichAlias</code>	Provide a vector of Aliases to only select these aliases for plotting.
<code>whichPeptide</code>	Filter what peptides to plot. This can either be a dataframe with a <code>ModifiedPeptide</code> peptide column, or a vector with the <code>ModifiedPeptide</code> sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the <code>FilterComparison</code> function.
<code>whichProtein</code>	Filter what proteins to plot. These are the IDs as presented in the <code>UniprotIDs</code> column in your GlycoDiveR data. This can either be a dataframe with a <code>UniprotIDs</code> column, or a vector with the <code>UniprotIDs</code> you want to keep.
<code>exactProteinMatch</code>	This is only relevant if you select for proteins using the <code>whichProtein</code> argument. When set to <code>TRUE</code> (default), your supplied <code>UniprotIDs</code> must be an exact match to the <code>UniprotIDs</code> in the dataframe. When set to <code>FALSE</code> , it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to <code>FALSE</code> .
<code>silent</code>	silence printed information (default = <code>FALSE</code>)

Value

The number of glycans per site

Examples

```
## Not run:
PlotGlycanPerSite(myData)
## End(Not run)
```

PlotGlycanSubcellularLocation

PlotGlycanSubcellularLocation

Description

Visualize the count or intensity of glycopeptides per subcellular localization.

Usage

```
PlotGlycanSubcellularLocation(
  input,
  summaryFunction = "count",
  zscoreCutoff = 0,
  type = "all",
  pointSize = c(3, 10),
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
summaryFunction	Either "count" to visualize the number of unique glycopeptides, or "intensity" to summarise by the log2 intensity of the unique glycopeptides.
zscoreCutoff	An absolute z-score cutoff.
type	Either "all" to plot for all glycopeptides, or specify a vector of glycotypes as in the GlycanType column (e.g., c("Sialylated", "Fucosylated"))
pointSize	What is the minimum and maximum point size.
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.

exactProteinMatch

This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.

silent

silence printed information

Value

a point plot.

Examples

```
## Not run:
PlotGlycanSubcellularLocation(mydata, summaryFunction = "intensity", zscoreCutoff = 2,
type = "Fucosylated", pointSize = c(1,10))

## End(Not run)
```

PlotGlycopeptideCount *PlotGlycoPeptideCount*

Description

Visualize the number of unique glycopeptides identified using a barplot.

Usage

```
PlotGlycopeptideCount(
  input,
  grouping = "condition",
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
grouping	grouping is "technicalReps", "biologicalReps", or "condition".
whichAlias	provide a vector of Aliases to only select these aliases for plotting.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.

exactProteinMatch

This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.

silent

silence printed information (default = FALSE)

Value

A graph showing the number of unique glycopeptides

Examples

```
## Not run:
PlotGlycoPSMCount(mydata)

PlotGlycoPSMCount(mydata, grouping = "technicalReps",
                  whichAlias = c("Alias1", "Alias2"))
## End(Not run)
```

PlotGlycoproteinCount *PlotGlycoproteinCount*

Description

Visualize the number of unique glycopeptides identified using a barplot.

Usage

```
PlotGlycoproteinCount(
  input,
  grouping = "condition",
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
grouping	grouping is "technicalReps", "biologicalReps", or "condition".
whichAlias	provide a vector of Aliases to only select these aliases for plotting.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.

whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
silent	silence printed information (default = FALSE)

Value

A graph showing the number of unique glycopeptides

Examples

```
## Not run:
PlotGlycoPSMCount(mydata)

PlotGlycoPSMCount(mydata, grouping = "technicalReps",
                  whichAlias = c("Alias1", "Alias2"))
## End(Not run)
```

PlotGlycoProteinGlycanNetwork

PlotGlycoproteinGlycanNetwork

Description

Creates a network where each node in the circle represents one glycan composition (e.g., H1N1, or H1N1A), connecting to a protein represented as a bar plot and sorted by number of identified glycan sites.

Usage

```
PlotGlycoProteinGlycanNetwork(
  input,
  edgeWidth = 1.5,
  verticeSize = c(5, 3),
  whichAlias = NULL,
  keepAllProteins = TRUE,
  highlight = "all",
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```


Arguments

input	Formatted data imported through a GlycoDiveR importer.
edgeWidth	Defines the linewidth of the edges.
verticeSize	Defines the size of the glycan and protein nodes, verticeSize = c(3,5) means a glycan node size of 3 and a protein node size of 5.
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
keepAllProteins	Works in concert with whichAlias. If you filter for specific aliases this determines whether to keep only the proteins found in the specific aliases or all proteins.
highlight	Specify what glycan category to highlight, or use "all" to highlight all.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
silent	silence printed information (default = FALSE).

Value

A glycoprotein to glycan network

Examples

```
## Not run:
PlotGlycoProteinGlycanNetwork(mydata)

PlotGlycoProteinGlycanNetwork(
  mydata,
  highlight = c("Sialyl", "Truncated"),
  edgeWidth = 3,
  verticeSize = c(3, 4)
)

## End(Not run)
```

PlotGlycoproteinRank *PlotGlycoproteinRank*

Description

Plot the glycoprotein ranks using a scatter plot. The protein intensity is the summed glycopeptide intensities of that proteins and then log10 transformed. The histogram shows the total intensity of all glycoproteins per bin.

Usage

```
PlotGlycoproteinRank(
  input,
  grouping = c("all", "technicalReps", "biologicalReps", "condition"),
  bins = 10,
  pointSize = 4,
  geneLabel = NULL,
  whichProtein = NULL,
  whichPeptide = NULL,
  exactProteinMatch = TRUE,
  whichAlias = NULL,
  histogramColor = "grey70",
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
grouping	Grouping is "all" "technicalReps", "biologicalReps", or "condition".
bins	The number of histogram bars.
pointSize	The point size in the scatter plot.
geneLabel	Specifu what genes to label. The values should correspond to the values in your imported data's Genes column. Supply like this: c("ORM1", "IGHM").
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
histogramColor	The color of the histogram bars.
silent	silence printed information

Value

A ggplot object.

Examples

```
## Not run:
PlotGlycoproteinRank(mydata, grouping = "condition")

PlotGlycoproteinRank(mydata, pointSize = 6,
  bins = 20, histogramColor = "#FFFFFF", grouping = "condition")

## End(Not run)
```

PlotGlycoPSMCount	<i>PlotGlycoPSMCount</i>
-------------------	--------------------------

Description

Visualize the number of identified glycoPSMs using a bargraph.

Usage

```
PlotGlycoPSMCount(
  input,
  grouping = "condition",
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
grouping	grouping is "technicalReps", "biologicalReps", or "condition".
whichAlias	provide a vector of Aliases to only select these aliases for plotting.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
silent	silence printed information (default = FALSE).

Value

A GlycoPSM graph

Examples

```
## Not run:
PlotGlycoPSMCount(mydata)

PlotGlycoPSMCount(mydata, grouping = "technicalReps", whichProtein = c("P01550"),
  exactProteinMatch = FALSE)
## End(Not run)
```

PlotGlycositeIdsPerRun

PlotGlycositeIdsPerRun

Description

Visualize site coverage per protein for different runs.

Usage

```
PlotGlycositeIdsPerRun(
  input,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  whichAlias = NULL,
  colorVec = c("#BAA5CC", "#32006e", "white"),
  plotGradient = c("white", "#44AA99"),
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
whichProtein	Filter what protein to plot (make sure to only provide one). These should be the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotID you want to keep.
exactProteinMatch	When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
whichAlias	provide a vector of Aliases to only select these aliases
colorVec	The color scheme used in the plot. Default = c("#6dc381", "pink", "#6761A8")
plotGradient	The fill gradient of the points.
silent	silence printed information (default = FALSE).

Value

A plot displaying the glycosites identified across different runs

Examples

```
## Not run:
PlotGlycositeIdsPerRun(mydata, whichProtein = "P17047")
## End(Not run)
```

PlotGlycositesVsGlycans

PlotGlycositesVsGlycans

Description

Plots the proteins by the number of glycosites on the proteins to the number of identified glycans identified on that protein.

Usage

```
PlotGlycositesVsGlycans(
  input,
  whichAlias = NULL,
  pointSize = 3,
  labelGlycositeCutoff = 0,
  labelGlycanCutoff = 0,
  maxOverlaps = 10,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  plotColor = c("#BAA5CC", "#32006e"),
  alpha = 0.5,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
whichAlias	provide a vector of Aliases to only select these aliases for plotting
pointSize	The size of the points.
labelGlycositeCutoff	The minimal number of glycosites found to label a protein (default = 0)
labelGlycanCutoff	The minimal number of glycans found to label a protein (default = 0)
maxOverlaps	The maximum number allowed label overlaps (default = 10)
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.

whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
plotColor	The color of plotted points.
alpha	The alpha of the plotted points.
silent	silence printed information (default = TRUE)

Value

A scatter plot comparing the number of glycosites to the number of glycans for each protein.

Examples

```
## Not run:
PlotGlycositesVsGlycans(mydata)

PlotGlycositesVsGlycans(mydata, plotColor = "black", alpha = 0.1, maxOverlaps = Inf)
## End(Not run)
```

PlotPCA

PlotPCA

Description

PCA plot based on PSM quantification. Missing values are imputed using kNN using the impute package.

Usage

```
PlotPCA(
  input,
  quantType = "normalized",
  dropNoQuant = TRUE,
  minPeptideCoverage = 0.5,
  thresholdMode = "total",
  grouping = "technicalReps",
  type = "glyco",
  returnType = c("PCA", "loadingsPlot", "loadingsData"),
  loadingsPlotLabels = 10,
  label = TRUE,
  whichProtein = NULL,
  whichPeptide = NULL,
  exactProteinMatch = TRUE,
  whichAlias = NULL,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
quantType	"normalized" for normalized intensity values, and "nonNormalized" for non-normalized intensity values.
dropNoQuant	Remove or keep PSMs without a quantitative value or have a quant of 0.
minPeptideCoverage	Works together with the thresholdMode argument. If thresholdMode = "total", provide a single numeric value. Values between 0 and 1 are interpreted as proportions (e.g., 0.1 = 10%), while values ≥ 1 are interpreted as absolute counts (e.g., 2 = at least two peptides identified). If thresholdMode = "group", provide a numeric vector of length two (e.g., c(0.1, 3)). The first value is interpreted in the same way as for "total" (proportion or absolute count). The second value specifies the minimum number of groups in which this threshold must be met.
thresholdMode	Character string specifying the filtering mode: "total" applies a global threshold across all data, while "group" identifies the threshold within groups.
grouping	Grouping is "technicalReps", "biologicalReps", or "condition".
type	Either "glyco" or "all".
returnType	Determines if the functions returns a PCA plot, a loadings plot, or the loadings plot data.
loadingsPlotLabels	Choose the number of points to label, based on the magnitude of each point calculated using $\sqrt{PC1^2 + PC2^2}$.
label	TRUE for showing Alias labels, FALSE for no labels.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
silent	silence printed information

Value

A ggplot PCA plot.

Examples

```
## Not run:
  PlotPCA(mydata)

  PlotPCA(mydata, quantType = "normalized", minPeptideCoverage = c(0.4, 2),
```

```

thresholdMode = "group", grouping = "biologicalReps", type = "all")

PlotPCA(mydata, type = "glyco", returnType = "loadingsPlot",
loadingsPlotLabels = 20)

## End(Not run)

```

PlotPeptideLength	<i>PlotPeptideLength</i>
-------------------	--------------------------

Description

Visualize the identified peptide lengths using a histogram.

Usage

```

PlotPeptideLength(
  input,
  type = "both",
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)

```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
type	Choose "both", "glyco", or "nonGlyco".
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
silent	silence printed information

Value

A ggplot graph.

Examples

```
## Not run:
PlotPeptideLength(mydata, input = "glyco")

## End(Not run)
```

PlotPSMCount	<i>PlotPSMCount</i>
--------------	---------------------

Description

Visualize the number of identified PSMs with a bar graph.

Usage

```
PlotPSMCount(
  input,
  grouping = "condition",
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
grouping	grouping is "technicalReps", "biologicalReps", or "condition".
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
silent	silence printed information (default = FALSE)

Value

the PSM count

Examples

```
## Not run:
PlotPSMCount(myData)

PlotPSMCount(mydata, grouping = "technicalReps")
## End(Not run)
```

PlotPSMsVsTime	<i>PlotPSMsVsTime</i>
----------------	-----------------------

Description

Visualize the number of (glyco)PSMs versus the retention time using a lineplot.

Usage

```
PlotPSMsVsTime(
  input,
  type = "all",
  binWidth = 5,
  lineWidth = 1,
  gradientLength = NA,
  plotColors = c("#bdbdbd", "#1b9e77"),
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
type	Choose between "allGlyco", "both", "glyco", "combined", "all", or supply a vector of glycan types, such as c("Multi", "nonGlyco", "Sialyl", "Complex/Hybrid", "Sialyl+Fucose", "Fucose", "Truncated", "High Mannose", "Paucimannose").
binWidth	the bin width used.
lineWidth	The width of the lines.
gradientLength	Define the LC gradient length. If no length is supplied the maximum PSM retention time is used.
plotColors	Define the colors of the non-glyco types. Default = c("#bdbdbd", "#1b9e77").
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.

exactProteinMatch

This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.

silent

silence printed information

Value

a lineplot

Examples

```
## Not run:
PlotPSMsVsTime(mydata)

PlotPSMsVsTime(mydata, type = "allGlyco", whichAlias = c("NP0-1-r1"),
  binWidth = 1, gradientLength = NA)

## End(Not run)
```

PlotPTMQuantification *PlotPTMQuantification*

Description

Visualize the glycan quantification of a single protein using a heatmap.

Usage

```
PlotPTMQuantification(
  input,
  whichProtein = NULL,
  whichPeptide = NULL,
  normalization = c("none", "ZScore"),
  removeSingletons = TRUE,
  collapseTechReps = FALSE,
  plotColors = c("#BAA5CC", "#32006e", "white"),
  heatmapColors = c("white", "#88CCEE", "#8877A1", "#882255", "#1a88c0"),
  exactProteinMatch = TRUE,
  whichAlias = NULL,
  lineWidth = 2,
  rowFontSize = 12,
  showRowNames = TRUE,
  clusterColumns = FALSE,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
whichProtein	Filter what protein to plot (make sure to only provide one). These should be the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotID you want to keep.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
normalization	"none" or "ZScore".
removeSingletons	Works in conjunction with normalization = "ZScore". If TRUE, it will remove any rows with fewer than 1 valid value. If set to TRUE, the singleton in a row will be colored light grey and the missing values white.
collapseTechReps	Collapse the technical replicates.
plotColors	Defines the colors of the barplot. Default: plotColors = c("#00394c", "#27b56e", "white").
heatmapColors	Defines the colors of the heatmap. Default: heatmapColors = c("white", "#27b56e", "#fdd835", "#d84315").
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
whichAlias	provide a vector of Aliases to only select these aliases for plotting
lineWidth	defines the black line around the heatmap cells. Use NA to remove the line
rowFontSize	font size for the row labels
showRowNames	set to TRUE or FALSE
clusterColumns	set to TRUE to cluster columns
silent	silence printed information

Value

The PTM quantification for a specific protein

Examples

```
## Not run:
PlotPTMQuantification(mydata, whichProtein = "P07361")

PlotPTMQuantification(mydata, whichProtein = "P07361", rowFontSize = 10,
                      showRowNames = FALSE, lineWidth = 0)
## End(Not run)
```

PlotQuantificationQC *PlotQuantificationQC*

Description

A boxplot showing Log2 intensity values either before or after normalization, or both.

Usage

```
PlotQuantificationQC(
  input,
  whichAlias = NULL,
  whichQuantification = "both",
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

input	Formatted data
whichAlias	provide a vector of Aliases to only select these aliases for plotting
whichQuantification	choose "both", "corrected", or "raw". Determines what quantification values to plot.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
silent	silence printed information (default = FALSE)

Value

Log2 intensity boxplots

Examples

```
## Not run:
PlotQuantificationQC(mydata, whichQuantification = "both")
## End(Not run)
```

PlotSiteGlycanComposition

PlotSiteGlycanComposition

Description

PlotSiteGlycanComposition

Usage

```
PlotSiteGlycanComposition(
  input,
  whichProtein,
  exactProteinMatch = TRUE,
  whichPeptide = NULL,
  whichAlias = NULL,
  domainColors = c("#BAA5CC", "#44AA99", "#a9a9a9", "#22031F", "#433E0E"),
  nodeColors = c("#32006e", "white"),
  horizontalPoints = 50,
  yCorrection = 0.25,
  yNudge = 1,
  boxSpacing = 0.075,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
whichAlias	A vector of aliases you want to include, corresponding to the aliases in your annotation input.
domainColors	The colors used for labeling the different domains.
nodeColors	The colors used for labeling the glycosites on the protein.
horizontalPoints	specify the point size. Larger number is smaller points.
yCorrection	move the points closer or further away. smaller values is less spacing between the points

yNudge	nudge the points up
boxSpacing	defines the spacing of the box around the points. Larger numbers mean more spacing
silent	TRUE if you want info to be printed, FALSE if not.

Value

A plot showing glyco site

Examples

```
## Not run:
PlotSiteGlycanComposition(input = mydata, whichProtein = "P01042",
  whichAlias = c("firstSample", "secondSample"))

## End(Not run)
```

PlotSiteGlycanCompositionBar

PlotSiteGlycanCompositionBar

Description

Visualize the glycan composition per site.

Usage

```
PlotSiteGlycanCompositionBar(
  input,
  grouping = "condition",
  scales = "fill",
  intensity = "raw",
  whichAlias = NULL,
  whichPeptide = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
grouping	Grouping is "none", "technicalReps", "biologicalReps", or "condition", or "all".
scales	Controls plot normalization, choose "fill" or "stack".
intensity	Raw log2, or log10 transformed intensity values.
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.

whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
silent	silence printed information

Value

A barplot.

Examples

```
## Not run:
PlotGlycanCompositionBar(mydatam scales = "stack")

## End(Not run)
```

PlotSiteQuantification

Barplot quantification per site

Description

A barplot showing site glycan quantification.

Usage

```
PlotSiteQuantification(
  input,
  whichProtein,
  site,
  whichPeptide = NULL,
  exactProteinMatch = TRUE,
  cutoff = NA,
  whichAlias = NULL,
  silent = FALSE
)
```

Arguments

input	The input data as imported through one of the GlycoDiveR importers.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
site	The site as in the ModificationID column.

whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
cutoff	An Intensity column, either in percentage or as an absolute number.
whichAlias	provide a vector of Aliases to only select these aliases for plotting
silent	TRUE if you want info to be printed, FALSE if not

Details

The mean of technical replicates is calculated and the input data filtered for the user specific glycan and peptide cutoffs.

Value

A graph.

Examples

```
## Not run:
PlotSiteQuantification(mydata, whichProtein = "P04004", site = "N243", cutoff = 1e9)

PlotSiteQuantification(mydata, whichProtein = "P04004", site = "N243", cutoff = "10%")
## End(Not run)
```

PlotUpSet	<i>PlotUpSet</i>
-----------	------------------

Description

Plot UpSet plots to find identification overlaps.

Usage

```
PlotUpSet(
  input,
  grouping = "condition",
  type = "glyco",
  level = "peptide",
  outputType = "plot",
  plotColor = "#32006e",
  whichAlias = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  nintersects = 40,
```

```

    whichPeptide = NULL,
    silent = FALSE
)

```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
grouping	Choose between "condition", "biologicalReps", and "technicalReps".
type	Choose between "glyco" or "all" to include only glycopeptides/proteins or all peptides/proteins.
level	Choose "peptide" or "protein".
outputType	Choose "plot" to return a visualization, and "data" to return the underlying data as a dataframe. Uses the nintersects argument to specify how many intersects to return.
plotColor	The color of the bars.
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
nintersects	The maximum number of intersects shown.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.
silent	TRUE if you want info to be printed, FALSE if not.

Value

An UpSet plot

Examples

```

## Not run:
PlotUpSet(mydata, grouping = "biologicalReps",
  type = "glyco", level = "protein", nintersects = 40)

## End(Not run)

```

PlotVennDiagram

*PlotVennDiagram***Description**

Creates a Venn diagram based for findings overlap in peptides, proteins, or glycans, either for glyco only or all. Only supports comparison between two or three groups. Two-group comparisons will be scaled proportionally, while three-way comparisons are not as this is mathematically only possible in a few cases.

Usage

```
PlotVennDiagram(
  input,
  groups,
  grouping = "condition",
  type = "glyco",
  level = c("peptide", "protein", "glycan"),
  whichAlias = NULL,
  whichProtein = NULL,
  exactProteinMatch = TRUE,
  whichPeptide = NULL,
  silent = FALSE
)
```

Arguments

input	Formatted data imported through a GlycoDiveR importer.
groups	Supply a vector of two or three groups of choice. This works in tandem with the grouping argument. If "technicalReps" is selected, grouping corresponds to the values in the Alias column. For "biologicalReps" this is the value in the Condition column and the BioReplicate column connected with a <code>_</code> . For example, Condition_1. For "condition" this is the value in the Condition column.
grouping	Choose "condition", "biologicalReps", or "technicalReps".
type	Choose either "glyco" to filter for only glycosylated PSMs, or select "all".
level	Compare either on the levels of "peptide", "protein", or "glycan".
whichAlias	Provide a vector of Aliases to only select these aliases for plotting.
whichProtein	Filter what proteins to plot. These are the IDs as presented in the UniprotIDs column in your GlycoDiveR data. This can either be a dataframe with a UniprotIDs column, or a vector with the UniprotIDs you want to keep.
exactProteinMatch	This is only relevant if you select for proteins using the whichProtein argument. When set to TRUE (default), your supplied UniprotIDs must be an exact match to the UniprotIDs in the dataframe. When set to FALSE, it will select non-exact matches. For example, "P61224" will only match to "P61224,P62834" when set to FALSE.
whichPeptide	Filter what peptides to plot. This can either be a dataframe with a ModifiedPeptide peptide column, or a vector with the ModifiedPeptide sequences that you want to keep. Inputted data with the comparison importer functions is directly usable, also after filtering using the FilterComparison function.

`silent` silence printed information (default = FALSE).

Value

A ggplot visualization.

Examples

```
## Not run:  
PlotVennDiagram(mydata, groups = c("Treated", "Untreated"), level = "glycan")  
  
PlotVennDiagram(mydata, groups = c("Treated_1", "Treated_2"),  
grouping = "biologicalReps", level = "protein")  
  
## End(Not run)
```

Index

* datasets

exampleData, [5](#)

ComputeMissingness, [3](#)

ComputeNumberOfGlycoforms, [4](#)

exampleData, [5](#)

FilterComparison, [6](#)

GetAnnotationTemplate, [7](#)

GetComparison, [7](#)

ImportByonic, [8](#)

ImportGlycanFinder, [9](#)

ImportMSFragger, [11](#)

ImportMSstatsAbundances, [13](#)

ImportMSstatsComparison, [13](#)

ImportPerseusAbundances, [14](#)

ImportPerseusComparison, [15](#)

ImporttpGlyco, [15](#)

ManageColorScheme, [17](#)

ManageGlycanCategories, [17](#)

ManageGlycanColors, [18](#)

ManageGlycanDatabase, [19](#)

ManageModificationDatabase, [20](#)

PlotComparisonVolcano, [21](#)

PlotCompletenessMatrix, [22](#)

PlotCV, [23](#)

PlotErrorVersusRt, [24](#)

PlotGlycanCompositionBar, [26](#)

PlotGlycanCompositionPie, [27](#)

PlotGlycansPerSite, [28](#)

PlotGlycanSubcellularLocation, [29](#)

PlotGlycopeptideCount, [30](#)

PlotGlycoproteinCount, [31](#)

PlotGlycoProteinGlycanNetwork, [32](#)

PlotGlycoproteinRank, [34](#)

PlotGlycoPSMCount, [35](#)

PlotGlycositeIdsPerRun, [36](#)

PlotGlycositesVsGlycans, [37](#)

PlotPCA, [38](#)

PlotPeptideLength, [40](#)

PlotPSMCount, [41](#)

PlotPSMsVsTime, [42](#)

PlotPTMQuantification, [43](#)

PlotQuantificationQC, [45](#)

PlotSiteGlycanComposition, [46](#)

PlotSiteGlycanCompositionBar, [47](#)

PlotSiteQuantification, [48](#)

PlotUpSet, [49](#)

PlotVennDiagram, [51](#)